# On-site test for Backend Engineer

***Important Note:*** You are not expected to write a full blown scalable web application. What we are looking for is a solid architecture, clean and elegant code. Also, please take into consideration that it is better to implement some parts bulletproof than to implement everything shallow and immature.

**General Guidelines:**

1. Please provide working, runnable code and a brief design document explaining what you did and how to test it. Proper integration tests are optional, but very welcome.
2. Feel free to search the Web for inspiration, but please do not copy code from anywhere - keep it 100% yours. If you use AI assistance, that is fine; just be ready to explain every line you submit.
3. Use Python 3.13+ as your language of implementation. FastAPI is recommended, but Flask or Django are acceptable if you prefer. Use type hints throughout and run a type checker (mypy/pyright/any other).
4. If you decide to use a relational database, use PostgreSQL (preferred). If you need a key/value store, use Redis.
5. Use a dependency/packaging tool such as Poetry, uv, or pip-tools with a pinned lockfile. Code should be formatted (black/ruff) and linted.
6. Please remember that the design should be ready for a heavy-traffic web application with many users and interactions.
7. You **don’t** need to implement a UI of any kind.

**Assignment**

You are to design and implement the API for a simplified Twitter-like social updates site.
The application shall support registration of new users and proper authentication of existing users.
Each user can post short text messages (140 chars).
Each user can follow other users and get a feed of their latest updates.
Each user can also get a global feed of all users.

**Implement an HTTP-based API that exposes the following calls:**

*Non-Authenticated (anonymous) access:*

SignUp(username, password)
SignIn(username, password) -> should return an authentication token for use in subsequent calls. Optionally: install an appropriate browser cookie that wraps the token.

*Authenticated access (Bearer/Cookie authentication):*

SignOut() -> should invalidate the authentication token and remove the cookie if it exists
PostMessage(message)
Follow(username_of_user_to_follow)
Unfollow(username_of_user_to_unfollow)
GetFeed(username_of_user_to_get_feed_of) -> returns the feed of any other user
GetFollowFeed() -> returns the feed of users I follow
GetGlobalFeed() -> returns the feed of all users

**Notes:**

Make sure all your API calls return appropriate HTTP status codes and a description of what went wrong, where appropriate.
We know it is not good practice from many standpoints, but for the sake of simplicity and to make it possible to use a browser in a simple manner, all calls may be implemented as HTTP GETs.
Provide an OpenAPI/Swagger schema (FastAPI gives this for free; for other frameworks, generate one).
Async vs. sync is your call - justify the choice in the design document.

**Bonus Points:**

“Dockerise” your solution using docker-compose.
**A lot of points**: document your work, in files that are not part of the code, it would be interesting to see your chain of thoughts.

**Good Luck!**